

Esta página ha sido traducida del inglés por la comunidad. Aprende más y únete a la comunidad de MDN Web Docs.

element.addEventListener

Resumen

`addEventListener()` Registra un evento a un objeto en específico. El [Objeto específico](#) ^(inglés) puede ser un simple [elemento](#) en un archivo, el mismo [documento](#) , una [ventana](#) o un [XMLHttpRequest](#) ^(inglés).

Para registrar más de un `eventListener`, puedes llamar `addEventListener()` para el mismo elemento pero con diferentes tipos de eventos o parámetros de captura.

Sintaxis

```
target.addEventListener(tipo, listener[, useCapture]);  
target.addEventListener(tipo, listener[, useCapture, wantsUntrusted  ]); // Gecko/Mozilla only
```

`tipo`

Una cadena representando el [tipo de evento](#) a escuchar.

`listener`

El objeto que recibe una notificación cuando un evento de el tipo especificado ocurre. Debe ser un objeto implementando la interfaz [EventListener](#) o solo una [function](#) ^(inglés) en JavaScript.

`useCapture` Opcional

Si es `true` , `useCapture` indica que el usuario desea iniciar la captura. Después de iniciar la captura, todos los eventos del tipo especificado serán lanzados al `listener` registrado antes de comenzar a ser controlados por algún `EventTarget` que esté por debajo en el árbol DOM del documento.

Nota: For event listeners attached to the event target; the event is in the target phase, rather than capturing and bubbling phases. Events in the target phase will trigger all listeners on an element regardless of the `useCapture` parameter.

Nota: `useCapture` became optional only in more recent versions of the major browsers; for example, it was not optional prior to Firefox 6. You should provide that parameter for broadest compatibility.

`wantsUntrusted`

If `true` , the listener receives synthetic events dispatched by web content (the default is `false` for chrome and `true` for regular web pages). This parameter is only available in Gecko and is mainly useful for the code in add-ons and the browser itself. See [Interaction between privileged and non-privileged pages](#) for an example.

Ejemplo

HTML

```
<!doctype html>  
<html>
```

```

<head>
  <title>DOM Event Example</title>

  <style>
    #t {
      border: 1px solid red;
    }
    #t1 {
      background-color: pink;
    }
  </style>

  <script>
    // Function to change the content of t2
    function modifyText() {
      var t2 = document.getElementById("t2");
      t2.firstChild.nodeValue = "three";
    }

    // Function to add event listener to t
    function load() {
      var el = document.getElementById("t");
      el.addEventListener("click", modifyText, false);
    }

    document.addEventListener("DOMContentLoaded", load, false);
  </script>
</head>
<body>
  <table id="t">
    <tr>
      <td id="t1">one</td>
    </tr>
    <tr>
      <td id="t2">two</td>
    </tr>
  </table>
</body>
</html>

```

[Ver en el JSFiddle](#)

En el ejemplo anterior, `modifyText()` es una listener para los eventos `click` registrados utilizando `addEventListener()`. Un click en cualquier parte de la tabla notificara al handler y ejecutara la función `modifyText()`.

Si quieres pasar parámetros a la función del listener, debes utilizar funciones anónimas.

HTML

```

<!doctype html>
<html>
  <head>
    <title>DOM Event Example</title>

    <style>
      #t {
        border: 1px solid red;
      }
      #t1 {
        background-color: pink;
      }
    </style>

    <script>
      // Function to change the content of t2

```